



React Krunal

What are Hooks?

Hooks in programming, especially in the context of frameworks like React, are functions that allow developers to use state and other React features without writing a class. They essentially let you use state and other React features inside functional components.

Each hook serves a specific purpose, enhancing functional components with capabilities previously reserved for class components, thereby promoting code reuse, modularity, and easier maintenance in React applications.

Advantages of Hooks:

- Hooks allow to easily manipulate the state of our functional component without needing to convert them into class components.
- Promotes code reuse
- Enhances modularity

useState:

`useState` is one of the built-in React hooks. `useState(0)` returns a tuple where the first parameter `count` is the current state of the counter and `setCounter` is the method that will allow us to update the counter's state.

Example:

```
const [value, setValue] = useState(0);
```

In this example, `useState` returns an array which has a value and a modifier to change that value.

useEffect

`useEffect` is a hook in React that allows you to perform side effects in function components. Side effects in React typically include data fetching, subscriptions, or manually changing the DOM in ways that React doesn't handle automatically.

```
useEffect(() => {  
  // Side effect code here  
}, [dependency]);
```

The second argument is an optional array of dependencies. These dependencies determine when the effect function should be re-run. If the dependencies array is empty, the effect will only run once after the initial render. If there are dependencies listed, the effect will run whenever any of those dependencies change. If you omit the second argument entirely, the effect will run after every render.

However, it's important to be mindful of performance considerations when using `useEffect` without dependencies, as running an effect after every render can potentially lead to performance issues if the effect is computationally expensive or causes unnecessary re-renders.

Render in React

"Render" in React refers to the process of converting React components and data into visible elements on the screen for the user to interact with.

```
const element = <h1>Hello, world!</h1>;  
// converts to
```

```
<h1>Hello, world!</h1>
```

Real DOM vs Virtual DOM

Real DOM:

- The Real DOM is the actual representation of the HTML elements in a tree-like structure.
- Any changes to the DOM trigger a reflow and repaint of the entire affected subtree, which can be computationally expensive, especially for large and complex web applications.
- Directly manipulating the Real DOM can lead to performance issues, particularly when there are frequent updates or changes.

Virtual DOM:

- The Virtual DOM is an abstraction of the Real DOM created by frameworks like React. It's a lightweight copy of the Real DOM that React keeps in memory.
- When there's a change in the state of a component in React, it doesn't immediately update the Real DOM. Instead, it updates the Virtual DOM first.
- After the Virtual DOM is updated, React performs a process called "reconciliation," where it compares the updated Virtual DOM with the previous version (using a process called "diffing") to identify the minimal set of changes needed to update the Real DOM.
- Because the Virtual DOM is a lightweight representation, updating it is faster than directly manipulating the Real DOM. Additionally, React only updates the Real DOM with the minimal necessary changes, improving performance.

useContext

`useContext` is particularly useful for accessing global data or configuration settings in your application without the need for prop drilling (passing props through multiple layers of components). It simplifies the process of sharing data across components and makes your code cleaner and more maintainable.

Here's how you can use the `useContext` hook:

```

// Creating a Context
import React, { createContext, useContext } from 'react';
const MyContext = createContext();

// Providing Context
function App() {
  const value = 'Hello from Context';
  return (
    <MyContext.Provider value={value}>
      <ChildComponent />
    </MyContext.Provider>
  );
}

// Consuming Context
function ChildComponent() {
  const contextValue = useContext(MyContext);
  return <div>{contextValue}</div>;
}

```

useMemo

The `useMemo` hook in React is used for memoizing the result of a function. It caches the result of the function and returns that cached result whenever the inputs to the function remain the same. This can help optimize performance by avoiding unnecessary re-computation of values.

```

import React, { useMemo } from 'react';

function MyComponent({ a, b }) {
  const result = useMemo(() => {
    // Expensive computation
    return a + b;
  }, [a, b]);
}

```

```
    return <div>{result}</div>;  
  }
```

useMemo is useful when you have expensive calculations or data processing that don't need to be re-computed on every render. By memoizing the result, you can avoid unnecessary work and improve performance.

useCallback

The useCallback hook in React is used to memoize a callback function, similar to useMemo but specifically for functions. It returns a memoized version of the callback function that only changes if one of the dependencies has changed. This can be useful for optimizing performance by avoiding unnecessary re-renders of components that rely on callback functions.

useRef

useRef is commonly used to access DOM elements directly. When you pass a ref attribute to a DOM element, React assigns the DOM element to the .current property of the useRef object.

```
import React, { useRef } from 'react';  
  
function MyComponent() {  
  const inputRef = useRef(null);  
  
  const handleClick = () => {  
    inputRef.current.focus(); // Focuses the input element  
  };  
  
  return (  
    <div>  
      <input ref={inputRef} type="text" />  
      <button onClick={handleClick}>Focus Input</button>  
    </div>  
  );  
}
```

```
    </div>
  );
}
```

SSR VS CSR

Server-Side Processing

With SSR, when a user requests a web page, the server processes the request by executing server-side code (e.g., PHP, Node.js, Python, etc.). This server-side code generates the HTML content of the requested page, including any dynamic data fetched from databases or other sources.

HTML Generation: The server-side code generates the complete HTML structure of the page, including the content, styles, and scripts. It typically involves using templates or frameworks to construct the HTML markup.

```
// SSRExample.js

import React from 'react';
import ReactDOMServer from 'react-dom/server';

function SSRExample() {
  return <h1>Hello, Server-Side Rendering!</h1>;
}

const html = ReactDOMServer.renderToString(<SSRExample />);
console.log(html);
```

Client-Side Rendering (CSR)

With CSR, the server sends a basic HTML document along with JavaScript and CSS files. The browser executes the JavaScript code, dynamically generating and updating the HTML content based on user interactions or data from the server. This approach can lead to faster initial page loads and a more interactive user

experience, as only the necessary data needs to be fetched from the server rather than entire HTML pages.

In CSR, the entire page load is not required every time the user navigates between pages or interacts with the application. Instead, only the necessary data and resources are fetched from the server, typically in the form of API requests or other asynchronous calls. This allows for a more dynamic and interactive user experience, as the browser can update specific parts of the page without having to reload the entire document.

However, it's important to note that the initial page load in CSR can still involve fetching a significant amount of resources, such as JavaScript files, CSS stylesheets, and initial data. Once these resources are loaded, subsequent interactions with the application can be much faster since they often only require fetching and updating specific data rather than reloading the entire page.

```
// CSRExample.js
import React from 'react';
import ReactDOM from 'react-dom';

function CSRExample() {
  return <h1>Hello, Client-Side Rendering!</h1>;
}

ReactDOM.render(<CSRExample />, document.getElementById('root'));
```

Lazy Loading

Lazy Loading in React JS optimizes the performance of React applications. The data is only rendered when visited or scrolled; it can be images, scripts, etc. Lazy loading helps to load the web page quickly and presents the limited content to the user that is needed for the interaction. Lazy loading can be more helpful in applications that have high-resolution images or data that alters the loading time of the application.

Technique in React that allows you to load components, modules, or assets asynchronously, improving the loading time of your application. React provides a

built-in `React.lazy()` method and `Suspense` component to achieve lazy loading.

```
const MyComponent = React.lazy(() => import('./MyComponent'));

<Suspense fallback={<div>Loading...</div>}>
  <MyComponent />
</Suspense>
```

Code Splitting

Code-splitting is an effective technique for optimizing the performance and efficiency of web applications, especially those with large code bases. By reducing the amount of code that needs to be loaded when a page first loads, code splitting can improve the user experience and make your application more responsive and fast.

Fetch API

The Fetch API provides a JavaScript interface for accessing and manipulating parts of the protocol, such as requests and responses. It also provides a global `fetch()` method that provides an easy, logical way to fetch resources asynchronously across the network.

Unlike `XMLHttpRequest` that is a callback-based API, Fetch is promise-based and provides a better alternative that can be easily used in service workers. Fetch also integrates advanced HTTP concepts such as CORS and other extensions to HTTP.

```
async function logMovies() {
  const response = await fetch("<http://example.com/movies.json>");
  const movies = await response.json();
  console.log(movies);
}
```

Here we are fetching a JSON file across the network, parsing it, and printing the data to the console. The simplest use of `fetch()` takes one argument — the path to

the resource you want to fetch — and does not directly return the JSON response body but instead returns a promise that resolves with a Response object.

The Response object, in turn, does not directly contain the actual JSON response body but is instead a representation of the entire HTTP response. So, to extract the JSON body content from the Response object, we use the `json()` method, which returns a second promise that resolves with the result of parsing the response body text as JSON.

The `fetch()` method can optionally accept a second parameter, an init object that allows you to control a number of different settings:

```
// Example POST method implementation:
async function postData(url = "", data = {}) {
  // Default options are marked with *
  const response = await fetch(url, {
    method: "POST", // *GET, POST, PUT, DELETE, etc.
    mode: "cors", // no-cors, *cors, same-origin
    cache: "no-cache", // *default, no-cache, reload, force-cache, only-if-cached
    credentials: "same-origin", // include, *same-origin, omit
    headers: {
      "Content-Type": "application/json",
      // 'Content-Type': 'application/x-www-form-urlencoded',
    },
    redirect: "follow", // manual, *follow, error
    referrerPolicy: "no-referrer", // no-referrer, *no-referrer-when-downgrade, origin, origin-when-cross-origin, same-origin, strict-origin, strict-origin-when-cross-origin, unsafe-url
    body: JSON.stringify(data), // body data type must match "Content-Type" header
  });
  return response.json(); // parses JSON response into native JavaScript objects
}

postData("<https://example.com/answer>", { answer: 42 }).then
```

```
((data) => {  
  console.log(data); // JSON data parsed by data.json() call  
});
```

```
// Sending request with credential included  
fetch("<https://example.com>", {  
  credentials: "include",  
});
```

Axios

Axios is a Javascript library used to make HTTP requests from node.js or XMLHttpRequests from the browser. It supports the Promise API that is native to JS ES6. Axios can be used to intercept HTTP requests and responses and enables client-side protection against XSRF. It also has the ability to cancel requests.

CORS

Cross-Origin Resource Sharing (CORS) is used to enable web pages hosted on one domain (origin) to make requests to resources hosted on a different domain (origin). Scenarios where this is common include:

- API Requests: When a frontend application (like a React app) needs to fetch data from an API hosted on a different server.
- CDNs: When loading assets (like fonts, scripts, or stylesheets) from a content delivery network (CDN) that is on a different domain.
- Third-Party Services: Integrating with third-party APIs or services that are hosted on a different domain.

CORS is configured on the server-side. Servers can specify which domains (origins) are allowed to make requests, what types of requests are allowed (GET, POST, etc.), and which headers can be included in the request.

Higher-Order Component (HOC)

A Higher-Order Component (HOC) in React is used to share common functionality between multiple components. It's like a function that takes a component as an argument and returns a new enhanced component with added features.

Example:

```
const EnhancedComponent = higherOrderComponent(WrappedComponent);
```

Custom Hooks

Custom hooks are a way to reuse stateful logic in React components. They are JavaScript functions whose names start with "use" and may call other hooks. Custom hooks allow you to extract component logic into reusable functions, which can be shared across multiple components.

Here's an example of a custom hook:

```
import { useState, useEffect } from 'react';

// Define a custom hook
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    async function fetchData() {
      try {
        const response = await fetch(url);
        const jsonData = await response.json();
        setData(jsonData);
        setLoading(false);
      } catch (error) {
        console.error('Error fetching data:', error);
        setLoading(false);
      }
    }
  });
}
```

```

    fetchData();

    // Cleanup function (optional)
    return () => {
      // Any cleanup logic can go here
    };
  }, [url]); // Dependency array ensures the effect runs only
              // when url changes

  return { data, loading };
}

// Usage example
function MyComponent() {
  const { data, loading } = useFetch('<https://api.example.com/data>');

  if (loading) {
    return <div>Loading...</div>;
  }

  return (
    <div>
      {data && data.map(item => (
        <div key={item.id}>{item.name}</div>
      ))}
    </div>
  );
}

```

Prop Drilling

Prop drilling occurs when you need to pass data from a parent component to a deeply nested child component through intermediate components that do not

need the data themselves. It involves passing props down through multiple levels of the component tree, even though intermediate components don't use those props directly. While prop drilling works, it can lead to verbose code and make components tightly coupled to their parent components.

Dependency on Intermediate Components: When passing props down through multiple levels of the component tree, each intermediate component becomes dependent on the structure of its parent and child components. If you change the prop name or structure in one component, you need to update all intermediate components that pass it down, creating a tight coupling between them.

Passing props to each component manually makes it complex.

Context Api

Here's how you can use the Context API in React:

```
// Import necessary modules from React
import React, { createContext, useState, useContext } from 'react';

// Create the context
const ThemeContext = createContext();

// Create a provider component
export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme(prevTheme => (prevTheme === 'light' ? 'dark' : 'light'));
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

```

    );
  };

  // Custom hook to consume the theme context
  export const useTheme = () => useContext(ThemeContext);

```

Passing Data from Child to Parent (Lifting up State)

Here's how you can pass data from a child component to a parent component in React:

```

// Import necessary modules from React
import React, { useState } from 'react';

// Parent component
const ParentComponent = () => {
  // Define state in the parent component to store the data passed from the child
  const [childData, setChildData] = useState('');

  // Function to receive data from the child component
  const handleChildData = (data) => {
    setChildData(data);
  };

  return (
    <div>
      <h1>Parent Component</h1>
      {/* Render the child component and pass the function as a prop */}
      <ChildComponent onData={handleChildData} />
      {/* Display the data received from the child */}
      {childData && <p>Data from Child: {childData}</p>}
    </div>
  );
}

```

```

};

// Child component
const ChildComponent = ({ onData }) => {
  // Define state in the child component to store the data
  const [inputValue, setInputValue] = useState('');

  // Function to handle changes in the input field and pass data to the parent
  const handleChange = (event) => {
    const value = event.target.value;
    setInputValue(value);
    // Call the function passed from the parent and pass the data
    onData(value);
  };

  return (
    <div>
      <h2>Child Component</h2>
      /* Input field to capture data */
      <input type="text" value={inputValue} onChange={handleChange} />
    </div>
  );
};

export default ParentComponent;

```

Other

You can use the following commands to set up a new React application:

```

npx create-react-app my-app
npm init react-app my-app

```


In a React application, `public/index.html` is the page template and `src/index.js` is the JavaScript entry point.

npm (Node Package Manager)

npm is the default package manager for Node.js. It is used to install, manage, and publish packages or modules for use in Node.js projects. It has commands for installing dependencies (`npm install`), updating packages (`npm update`), and running scripts defined in the `package.json` file (`npm run`). npm installs packages globally (accessible from anywhere in your system) or locally (accessible only in the project directory).

npx (Node Package Executer)

npx is a tool introduced with npm 5.2.0 that comes bundled with npm. It is used to execute Node.js packages without installing them globally. npx runs the given command either from a local `node_modules/.bin` directory or downloads and installs the package in a temporary directory before executing it. It's particularly useful for running commands from packages that you don't want to install globally, such as one-off scripts or CLI tools.

The `robots.txt` file is used to make web crawling easier.